电子科技大学
University of Electronic Science and Technology of China

工业软件创新训练II

任务描述

任务描述

- 任务一：熟悉芯片前端设计流程，理解逻辑综合原理和过程。
- 任务二：研读Yosys代码，理解核心数据结构和优化算法。
- 任务三：设计和实现一个简单的Verilog编译器，并将其用于Yosys 系统。
- 任务四：使用dot语言绘制电路网表图。
- 任务五：设计和实现至少两种优化方法，如共享子表达式优化、常量传播，并将其用于Yosys 系统。

任务一：熟悉芯片前端设计流程，理解逻辑综合过程

学习内容：

- 理解逻辑综合原理和过程。
- 使用开源逻辑综合Yosys工具，测试一个从RTL代码到门级网表的芯片前端开发流程。

  测试代码见1_Yosys工具

  测试命令：MyVSYosys.exe -s synth.ys

  参考文档：逻辑综合流程.pdf

任务二：研读Yosys代码

学习内容：

- 理解核心数据结构

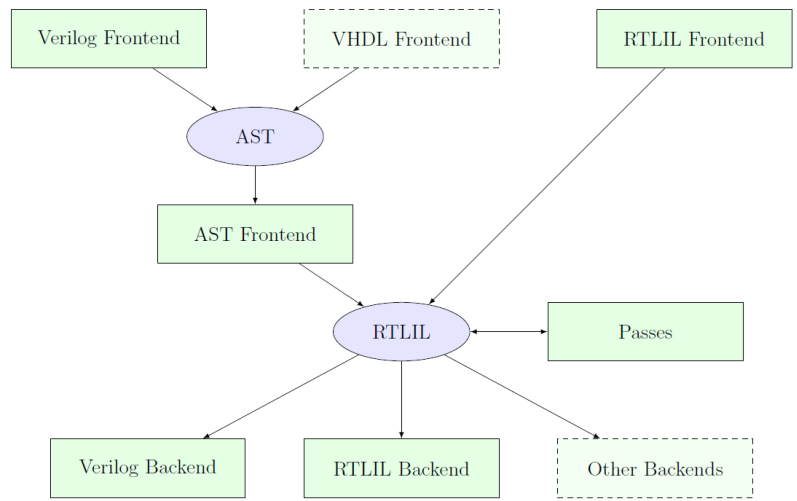


Fig. 4.1: Yosys simplified data flow (ellipses: data structures, rectangles: program modules)

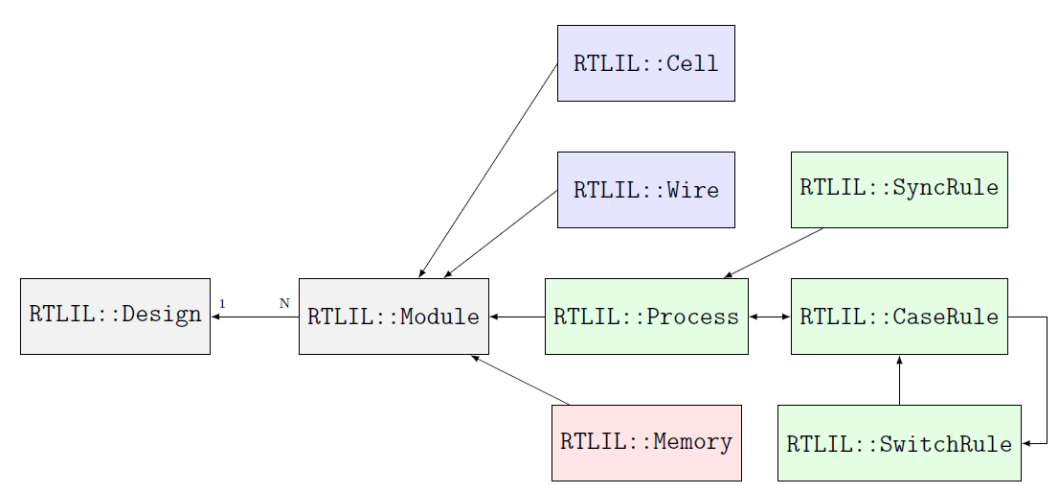


Fig. 4.2: Simplified RTLIL Entity-Relationship Diagram

参考：

https://yosys.readthedocs.io/_/downloads/en/latest/pdf/

https://blog.csdn.net/qq_44850725/article/details/126175561

代码： YosysCode /MyVSYosys-pub.rar

任务二：研读Yosys代码

学习内容：

- 重点优化算法的原理和步骤

优化算子	对应命令	核心逻辑	代码位置
常量折叠	opt_expr	如果与门（$and）输入全是常数，直接计算出结果并替换 1 2。	passes/opt/opt_expr.cc
死代码消除	opt_clean	找出输出没被连接的cell，直接删掉 1 2。	passes/opt/opt_clean.cc
常量传播(简化版)	opt_expr的子功能	与门一个输入是常数1，另一个是信号a，则简化为缓冲器(buffer) 1 4。	passes/opt/opt_expr.cc
冗余单元合并	opt_merge	找两个加法器，输入都是a和b，合并成一个，输出分给两处 1 2。	passes/opt/opt_merge.cc
多路选择器简化	opt_muxtree	y = a ? (a ? 1 : 2) : 3 → 内层MUX条件永远矛盾，直接替换为常数1 1 2。	passes/opt/opt_muxtree.cc
资源共享	share	识别电路中具有相同输入的功能单元（如加法器、乘法器等），合并共享以减少硬件资源消耗 1 2 4。	passes/opt/share.cc

任务三：设计和实现一个简单的Verilog编译器

学习内容：

- 3_1_Verilog编译与仿真工具使用

  见编译与仿真工具使用.docx

- 3_2_多变量表达式编译问题

  见多变量表达式编译问题.docx

编程：

开发一个简单Verilog解析器（主要支持assign和=的），并作为Yosys前端使用（即替换read_verilog）。

编译器例子：见demo1_Verilog编译.rar

任务三：设计和实现一个简单的Verilog编译器

例子程序： demo1_Verilog编译.rar

该程序实现基本词法和语法分析，能解析简单的verilog语法代码。

可改善的问题：

1. 输入不是verilog文件。
2. 没有将端口信息、门信息提取出来。
3. 生成的语法树结点，但没有将这些结点连成树结构。
4. 词法分析与语法分析交织进行，复杂度高，可考虑先对全体做词法分析，生成一组token，再做语法分析。
5. 仅支持解析assign语句。

任务三：设计和实现一个简单的Verilog编译器

设计思路：Yosys内部使用一种名为RTLIL（Register Transfer Level Internal Language）的数据结构来存储设计。一个标准的read_verilog前端所做的工作就是：解析文本文件 -> 构建抽象语法树(AST) -> 将AST转换为RTLIL结构并追加到当前设计中。

步骤1：词法分析（Lexical Analysis）

识别关键词（如module、assign、input、output）、操作符（如=、&、|、^）、标识符（如变量名a）、数字（如1'b0、32'hFF）以及符号（如分号;、括号()）。

步骤2：语法分析（Syntax Analysis）

构建AST节点来表示Module、Port、Wire和Assign

步骤3：Yosys前端接口与RTLIL生成

任务四：使用dot语言绘制电路网表图

学习内容：电路网表图绘制

见电路网表图绘制.docx

参考例子程序： demo2_电路网表图绘制.rar

该程序完成存储中间数据结构，并实现网表图形绘制。

可改善的问题：

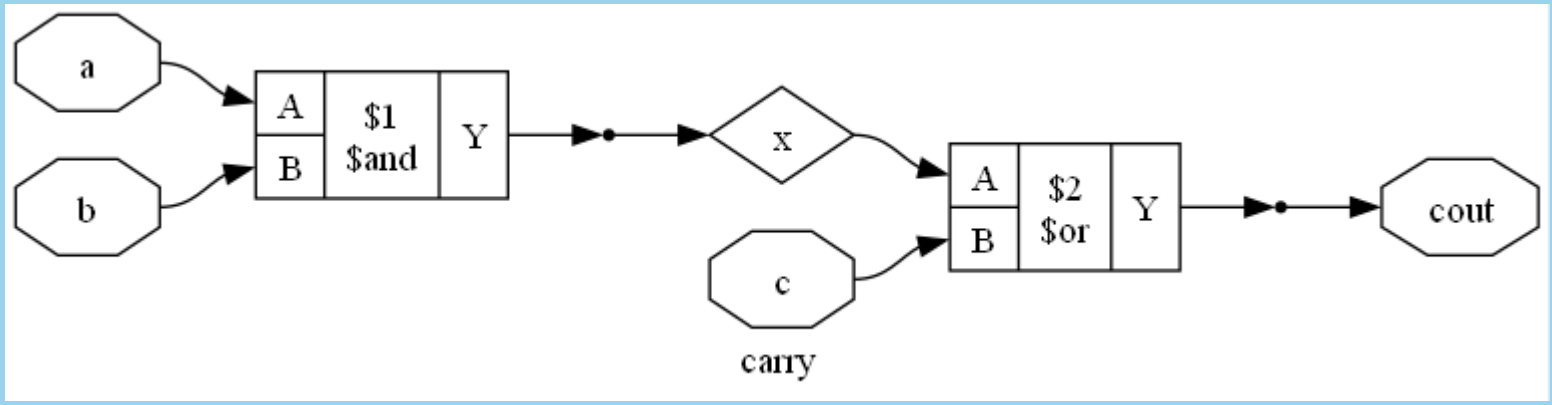
1. 仅支持assign x = a & b;的形式，多变量表达式，如assign x = a & b & c程序报错。
2. 不支持一元运算，如 assign x = ！ a;或 assign x = ！ a & b;
3. 没有考虑符号优先级，也不支持括号。

任务四：使用dot语言绘制电路网表图程序输入输出范例

1. 输入test1.v

```
module carry(input a,b,c,output cout);
wire x;
assign x = a & b;
assign cout = x | c;
endmodule
```

2. 输出图形文件



任务五：设计和实现优化方法

学习内容：

电路综合优化概述.pdf

共享子表达式优化

见共享子表达式优化实验与对标.docx

任务五：设计和实现优化方法

1. 设计共享子表达式(Sub-Expressions)优化方法，完成调用、测试，将其用于Yosys 系统。

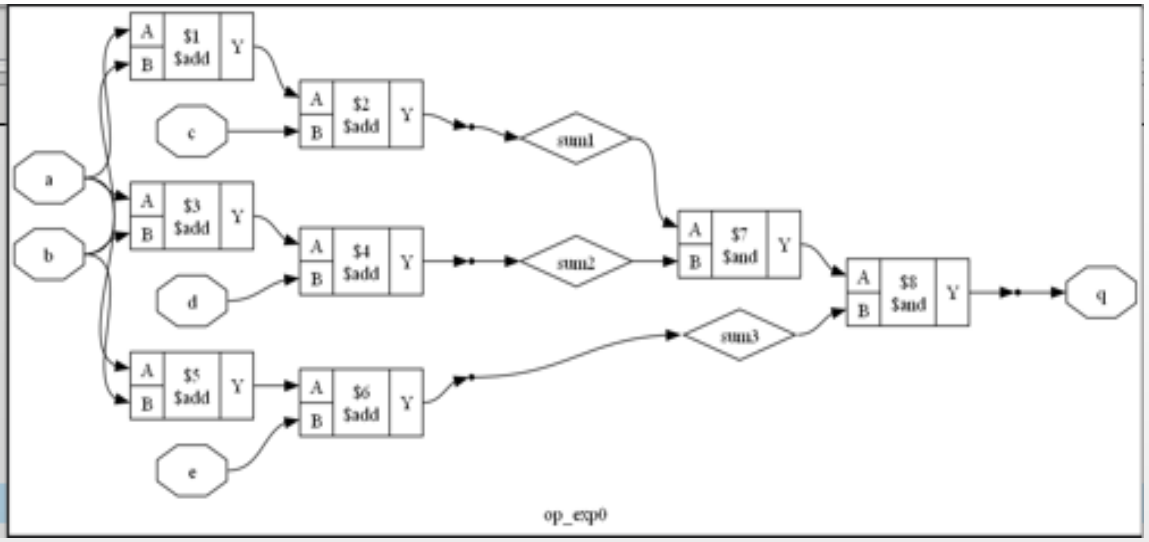
(选做) 若输入文件为op_exp0_bracket.v，设计并实现共享子表达式优化方法，并如问题1完成调用和测试等。

2. 设计并实现其他优化方法。

任务五：共享子表达式(Sub-Expressions)优化例子

op_exp0.v

```
1  module op_exp0(a,b,c,d,e,q);
2  input a,b,c,d,e;
3  output q;
4  wire sum1,sum2;
5  assign sum1 = a + b + c;
6  assign sum2 = a + b + d;
7  assign sum3 = a + b + e;
8  assign q = sum1 & sum2 & sum3;
9  endmodule
10
```



op_exp1.v

```
1  module op_exp1(a,b,c,d,e,q);
2  input a,b,c,d,e;
3  output q;
4  wire sum1,sum2,temp;
5  assign temp = a + b;
6  assign sum1 = temp + c;
7  assign sum2 = temp + d;
8  assign sum3 = temp + e;
9  assign q = sum1 & sum2 & sum3;
10 endmodule
```

